

ALP

FICHA DE EXERCÍCIOS ATIVIDADE LETIVA

Programação Orientada a Objetos

UNIDADE CURRICULAR

Ficha 7 – Autenticação e Controlo de Acesso

FICHA

Crie uma pasta para esta ficha (sugestão de nome: `F07_SoundBase Auth`).

Dentro dessa pasta, coloque os ficheiros disponíveis na [solução da Ficha 06](#).

Substitua o `index.html` e adicione `renderAppScreen()` ao `app-view.js` com os ficheiros fornecidos no Moodle.

Instale as dependências com `npm install` antes de começar.

Esta ficha introduz autenticação, controlo de acesso por role e arquitetura SPA com injeção dinâmica de HTML. Os ecrãs são construídos e injetados no elemento `#app` – cada view exporta funções `render*()` para construir o ecrã e funções `bind*()` para registar os event listeners. As funções `bind*()` devem ser sempre chamadas imediatamente após a respetiva `render*()`. Leia cada exercício na totalidade antes de começar.

Configuração do Servidor Mock

- 1) Instale as dependências necessárias: `npm install json-server@0.17.4 json-server-auth@2.1.0`
- 2) Crie o ficheiro `db.json` na raiz da pasta com a seguinte estrutura:

- Um array `"users"` vazio.
- Um array `"tracks"` com as faixas migradas de `data/catalogue.js`.
- Cada faixa deve ter um campo `id` inteiro único.

- 3) Crie o ficheiro `routes.json` na raiz da pasta com o seguinte conteúdo:

```
{
  "users": 600,
  "/tracks*": "/660/tracks$1"
}
```

- A regra `600` em `users` garante que cada utilizador só consegue ler e escrever os seus próprios dados.
- A regra `/660/tracks` garante que qualquer acesso a `/tracks` – leitura ou escrita – exige autenticação.
- Um pedido sem token válido recebe `401 Unauthorized`.

- 4) No `package.json`, adicione o seguinte script de arranque:

- "start": "json-server-auth db.json -port 3000"
- Inicie o servidor com `npm start` e confirme que `http://localhost:3000/tracks` devolve 401 num browser (sem token) e que as faixas são devolvidas ao chamar com um token válido.

Nota: O `json-server-auth` expõe automaticamente os endpoints `POST /register` e `POST /login`. O campo `role` no registo permite diferenciar utilizadores comuns de administradores - guarde pelo menos um utilizador com `role: 'admin'` para testar o painel de administração. A restrição por `role` é gerida no cliente; o servidor apenas valida se o utilizador está autenticado.

Camada de Serviço

5) Crie o ficheiro `js/services/service.js`. Implemente e exporte as seguintes funções:

- `register(email, password)` - faz `POST /register` com `{ email, password, role: 'user' }` e devolve `{ ok }`.
- `login(email, password)` - faz `POST /login` e devolve `{ ok, token, user }` em caso de sucesso, ou `{ ok: false }` em caso de erro. O servidor responde com `accessToken` - mapeie internamente para `token` antes de devolver, de modo que os controllers nunca dependam do formato da resposta do servidor.
- `getTracks()` - faz `GET /tracks` e devolve o array de objetos.
- `addTrack(data)` - faz `POST /tracks` com autenticação e devolve `{ ok }`.
- `deleteTrack(id)` - faz `DELETE /tracks/:id` com autenticação e devolve `{ ok }`.

Nota: O token de autenticação deve ser lido de `localStorage` dentro de uma função privada `authHeaders()`. Os controllers nunca devem conhecer a estrutura dos headers nem a URL base do servidor.

Migração dos Dados

- 6) Em `app-controller.js`, substitua a importação do catálogo estático por uma chamada a `getTracks()` do serviço.
- Torne `init()` assíncrona e aguarde os dados antes de registar os event listeners e renderizar.
- 7) Em `models/Tracks.js`, atualize o método estático `fromObject()` para preservar `obj.id` e `obj.plays` do objeto recebido.
- Confirme que a aplicação se comporta exatamente como antes da migração.

Nota: Compare sempre ids com `String(t.id) === String(id)` para evitar inconsistências entre inteiros (JSON Server) e strings (atributos `data-id` do HTML).

Registo

8) Crie o ficheiro `js/views/auth-view.js` com as seguintes funções exportadas:

- `renderRegisterScreen()` - injeta no `#app` um formulário de registo com:

- o Um `<form id="register-form">` com `<input type="email" id="register-email">`, `<input type="password" id="register-password">` e `<button type="submit" id="btn-register">` com o texto "Sign Up".
 - o Um `<p id="register-error" class="error-msg">` inicialmente vazio.
 - o Um parágrafo com `<span id="link-to-login">` e o texto "Already have na account".
 - `getRegisterFormData()` - devolve { email, password } do formulário de registo.
 - `setRegisterError(message)` - escreve a mensagem em #register-error; se vazia, limpa o parágrafo.
 - `bindRegisterSubmit(handler)` - regista o handler no evento submit do #register-form, com `e.preventDefault()`.
 - `bindLinkToLogin(handler)` - regista o handler no clique de #link-to-login.
- 9) Crie o ficheiro `js/controllers/auth-controller.js` e implemente:
- Uma função privada `handleRegister()` que lê os dados do formulário, chama `register()` do serviço e, em caso de sucesso, chama `showLogin()`; em caso de falha, exibe o erro com `setRegisterError()`. Envolve a chamada em `try/catch` para tratar falhas de rede.
 - Uma função privada `showRegister()` que chama `renderRegisterScreen()` seguida de `bindRegisterSubmit(handleRegister)` e `bindLinkToLogin(showLogin)`.
 - Uma função exportada `initAuth()` que verifica se já existe sessão ativa em `localStorage`; se existir, chama `startApp(user)` diretamente; caso contrário, chama `showLogin()`.
- 10) Em `js/app.js`, substitua a chamada a `init()` por `initAuth()`. Como `showLogin()` ainda não existe, substitua temporariamente essa chamada por `showRegister()` para testar o fluxo de registo. Confirme que o formulário é apresentado e que é possível registar um utilizador.

Login

- 11) Em `auth-view.js`, adicione e exporte as seguintes funções:
- `renderLoginScreen()` - injeta no #app um formulário de login com:
 - o Um `<form id="login-form">` com `<input type="email" id="login-email">`, `<input type="password" id="login-password">` e `<button type="submit" id="btn-login">` com o texto "Log in".
 - o Um `<p id="login-error" class="error-msg">` inicialmente vazio.
 - o Um parágrafo com `<span id="link-to-register">` e o texto "Create na account".
 - `getLoginFormData()` - devolve { email, password } do formulário de login.
 - `setLoginError(message)` - escreve a mensagem em #login-error; se vazia, limpa-o.
 - `bindLoginSubmit(handler)` - regista o handler no evento submit do #login-form, com `e.preventDefault()`.
 - `bindLinkToRegister(handler)` - regista o handler no clique de #link-to-register.
- 12) Em `auth-controller.js`, implemente a função privada `handleLogin()` que:

- Lê os dados com `getLoginFormData()` e chama `login()` do serviço.
 - Em caso de sucesso, guarda o token e o utilizador em `localStorage` (chaves “token” e “user”) e chama `startApp(user)`.
 - Em caso de erro, exibe uma mensagem com `setLoginError()`.
 - Envolve a chamada em `try/catch` para tratar falhas de rede.
- 13) Implemente a função privada `showLogin()` que chama `renderLoginScreen()` seguida de `bindLoginSubmit(handleLogin)` e `bindLinkToRegister(showRegister)`.
- Remova a substituição temporária do exercício 9 e confirme que a aplicação arranca no ecrã de login, com a navegação funcional entre os dois ecrãs.

Nota: Cada vez que um ecrã é renderizado, o DOM anterior é substituído e os event listeners anteriores são destruídos automaticamente – não é necessários removê-los. Por isso, `bind*()` deve ser sempre chamado depois de `render*()`.

Controlo de Acesso

- 14) Em `auth-controller.js`, implemente e exporte a função assíncrona `startApp(user)` que:
- Chama `renderAppScreen()` (importada de `app-view.js`).
 - Regista o handler de logout com `bindLogout()` (importado de `app-view.js`): ao clicar, remove o token e o utilizador do `localStorage` e chama `showLogin()`.
 - Se `user.role === 'admin'`, importa dinamicamente `admin-controller.js` e chama `init()`.
 - Caso contrário, importa `app-controller.js` e chama `init()`.

Nota: `bindLogout()` deve ser adicionado por si a `app-view.js` – é um simples `addEventListener` no `#btn-logout`, que já existe no template de `renderAppScreen()` fornecido. `renderAppScreen()` já está importada na ficha fornecida; `bindLogout()` é o único elemento que precisa de adicionar.

Painel de Administrador

- 15) Crie o ficheiro `js/views/admin-view.js` com as seguintes funções exportadas:
- `renderAdminPanel()` – injeta no `#app` um painel de administração com:
 - Um `<form id="add-track-form">` com campos de texto para `title`, `artist`, `genre` e `duration` (ids prefixados com `add-`) e um `<button type="submit" id="btn-add-track">` com o texto “Add Track”.
 - Uma tabela com cabeçalhos `Title`, `Artist`, `Genre`, `Duration` e `Actions`, e um `<tbody id="admin-track-list">`.
 - `renderAdminTrackList(tracks)` – renderiza o array de faixas na tabela, com um botão “Remove” por linha com `data-id` definido.
 - `getAddTrackFormData()` – devolve `{ title, artist, genre, duration, plays: 0 }`.
 - `clearAddTrackForm()` – limpa os quatro campos do formulário.

- `bindAddTrackSubmit(handler)` – regista o handler no evento submit do `#add-track-form`, com `e.preventDefault()`.
- `bindRemoveTrack(handler)` – regista um listener por delegação no `#admin-track-list` que chama `handler(id)` ao clicar num botão “Remove”.

16) Crie o ficheiro `js/controllers/admin-controller.js` com uma função exportada `init()` que:

- Chama `renderAdminPanel()`.
- Chama `getTracks()` do serviço e renderiza a lista com `renderAdminTrackList()`.
- Usa `bindAddTrackSubmit` para registar um handler que lê os dados do formulário, chama `addTrack()` do serviço e, em caso de sucesso, volta a carregar e renderizar a lista e limpa o formulário.
- Usa `bindRemoveTrack` para registar um handler que chama `deleteTrack(id)` do serviço e, em caso de sucesso, volta a carregar e renderizar a lista.

Desafio: Routing com Proteção de Rotas

A navegação atual é controlada por chamadas diretas a `show*()` dispersas pelos controllers. Esta abordagem não escala: qualquer controller pode navegar para qualquer ecrã e não existe um mecanismo central para proteger rotas que exigem autenticação.

- Crie o ficheiro `js/Router.js` e implemente uma classe `Router` com:
 - Um campo privado `#routes` (objeto vazio) para armazenar as rotas registadas.
 - Um método `add(hash, handler, guard = null)` que regista a rota e devolve `this` para permitir encadeamento.
 - Um método privado `#handleRoute()` que lê `window.location.hash`, verifica o `guard` (redirecionando para `'#/login'` se falhar) e executa o handler.
 - Um método `navigate(hash)` que atualiza `window.location.hash`.
 - Um método `listen()` que regista um listener no evento `hashchange` e chama `#handleRoute()` para tratar a rota inicial.
- Em `auth-controller.js`, exporte também:
 - `isAuthenticated()` – devolve `true` se existir token em `localStorage`.
 - `isAdmin()` – devolve `true` se o utilizador guardado em `localStorage` tiver `role === 'admin'`.
- Em `app.js`, substitua a lógica atual pela configuração seguinte e adapte os handler de `show*()` para usarem `navigate()` em vez de chamadas diretas:


```
const router = new Router();
const navigate = router.navigate.bind(router);

router
  .add('#/login', () => showLogin())
```

```
.add('#/register', () => showRegister())
.add('#/app', async () => { const u = JSON.parse(localStorage.getItem('user')); await startApp(u); }, isAuthenticated)
.add('#/admin', async () => { const u = JSON.parse(localStorage.getItem('user')); await startApp(u); }, isAdmin);

const stored = localStorage.getItem('user');
if (stored) navigate(JSON.parse(stored).role === 'admin' ? '#/admin' : '#/app');
else navigate('#/login');

router.listen();
```

- Confirme que um refresh de página repõe a sessão ativa sem redirecionar para o login.